

Introduction to Fortran

Victor Pisinger

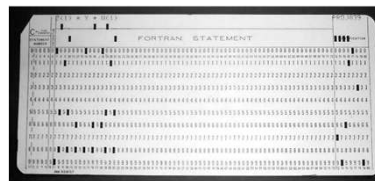
1

What is Fortran?

- A very old programming language
 - First version released by IBM in 1957 for their IBM 704 mainframe



LLNL(on flickr.com)



The Craft of Coding (on wordpress.com)

- Still updated and maintained, still widely used
 - Latest standard published in 2023
- Used especially for demanding scientific computation

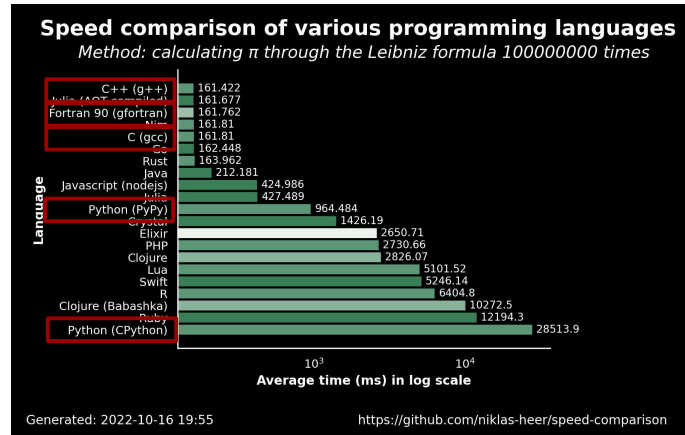


wikipedia.org

2

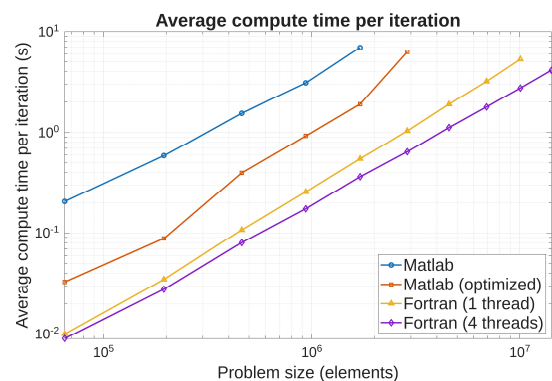
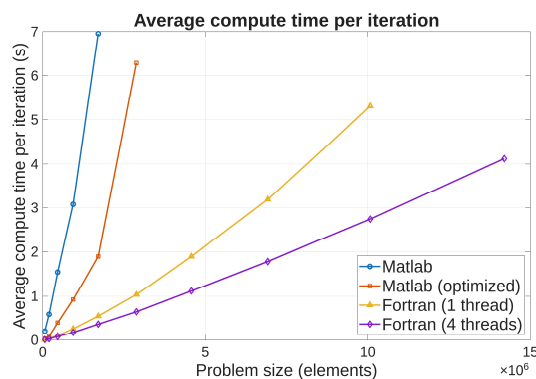
Why Use Fortran in Finite Element Methods?

- Every programming language has its uses!
- For simulations – We are interested in computation SPEED!
- Choice between C, C++ and Fortran
- Fortran is chosen due to learning curve and syntax similarities to MATLAB
- Easy to make sufficiently fast code
- Extended use in commercial FE software



3

Truss Topology Optimization (week 4)

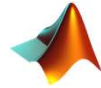


4

Some key differences from MATLAB



- Fortran is statically compiled
 - Write the code, compile the code, run the code
 - Creates separate executables. Once you are done programming you are done with Fortran
- Fortran is strongly typed
 - Variables must be declared with a data type before use
- Fortran is case-insensitive: $x = X$
- Fortran is “bare bones”
 - Has some hundreds built-in functions



- MATLAB is just-in-time compiled
 - Write the code, run the code
 - Does not create executables. MATLAB must always be opened to run the code
- MATLAB is weakly typed
 - Any data type can be put in any variable (almost)
- MATLAB is case-sensitive: $x \neq X$
- MATLAB is feature-rich
 - Has many expansions and thousands of built-in functions

5

How is Fortran so Fast?

- The Fortran language gives the compiler unusually strong information about **memory layout, aliasing, and numerical loops**.
- This in return allows the compiler to generate very efficient machine code
- This is partly due to Fortrans simple strongly typed syntax and design around numerical arrays and loops
- Example:

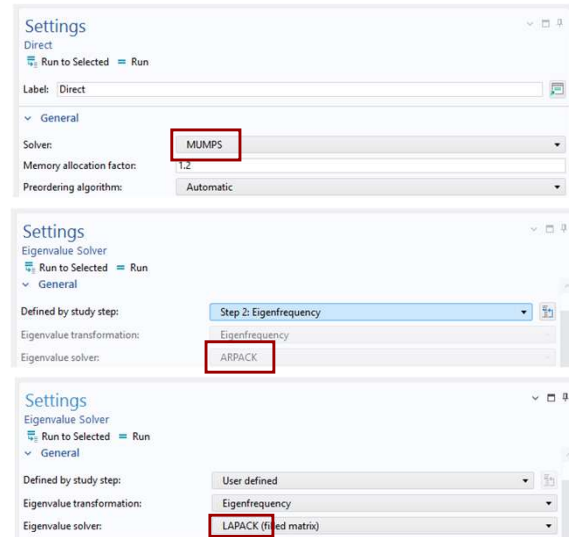
```
do i = 1, n
  c(i) = a(i) + b(i)
end do
```

Can be heavily optimized to using vectorization and memory alignment.

6

Fortran has an extensive function-base for High Performance Computing (HPC):

- Solvers and linear algebra:
 - BLAS
 - MUMPS
 - LAPACK
 - ARPACK
- Commercial FEA Programs:
 - ANSYS
 - Abaqus
 - (COMSOL)



7

Fortran Syntax Walkthrough

8

The Fortran language: Program structure

Try me!

```
program AddTwoNumbers
  implicit none
```

```
! Variable declarations
```

```
integer :: a, b, c
```

```
! Add two numbers
```

```
a = 5
```

```
b = 10
```

```
c = a + b
```

```
! Print result to screen
```

```
print*, 'The sum is ', c
```

```
end program
```

Start and name of the program

Comments start with !

Variable type declarations before use

Assignment and addition

Note that semi-colons ; are not used

Print result to screen

The asterisk * will be discussed later

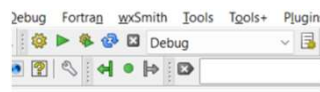
End of program

How to program Fortran: The steps required

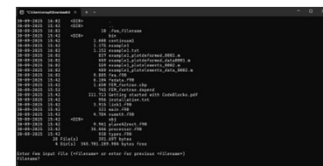
Write some code



Compile to executable



Run program



- Fortran files are plain text files (called source files)
- Any text editor will do (e.g. Code::Blocks, Notepad++, VS Code)

- A compiler turns the plain text files containing Fortran code into an executable
- Many compilers are available (We use GNU Compiler "gfortran")
- Options for optimization which can dramatically speed up your code

- Typically, from a shell (command window)

The Fortran language: Built-in data types

- **logical** – boolean values `.true.` or `.false.`
 - **integer** – integer values
 - **real** – floating point values
 - **complex** – complex numbers
 - **character** – characters or strings (text variables)
- All *number* variables can be given a **kind** specifier, which sets the precision (number of bytes) used to store the number (defaults to **kind** = 4)
 - `integer(kind = 8) :: x`

We have defined working precision (wp):

– `integer(wp) :: x`

11

The Fortran Language: Operators: Arithmetic

- The basic arithmetic operators are

Description	Fortran operator	MATLAB operator
Addition	+	+
Subtraction	-	-
Multiplication	*	*
Division	/	/
Exponential	**	^

These one always causes trouble!

12

The Fortran Language: Operators: Relational

- The relational operators are

Description	Fortran operator	MATLAB operator
Equal	<code>==</code> or <code>.eq.</code>	<code>==</code>
Not equal	<code>/=</code> or <code>.ne.</code>	<code>~=</code>
Greater than	<code>></code> or <code>.gt.</code>	<code>></code>
Less than	<code><</code> or <code>.lt.</code>	<code><</code>
Greater or equal	<code>>=</code> or <code>.ge.</code>	<code>>=</code>
Less or equal	<code><=</code> or <code>.le.</code>	<code><=</code>

The Fortran Language: Operators: Logical

- The relational operators are

Description	Fortran operator	MATLAB operator
And	<code>.and.</code>	<code>&&</code>
Or	<code>.or.</code>	<code> </code>
Not	<code>.not.</code>	<code>~</code>
Equivalent	<code>.eqv.</code>	<code>==</code>
Not equivalent	<code>.neqv.</code>	<code>~=</code>

The Fortran language: Flow control: **do** loops

```
do
  ! Do stuff forever
end do

do i = 1, 10
  ! Do some stuff 10 times
end do

do i = n, 1, -1
  ! Count down from n
end do

do while (x > 0)
  ! Do stuff until x <= 0
end do
```

Endless loop

Remember to initialize loop variable
before use (**integer** :: i)

Finite loop with predetermined length

Finite loop with variable length and
specified increment size

Please ensure it stop condition is met

Conditional loop

15

The Fortran language: Flow control: **if**

```
if (a == 1) then
  ! Do some stuff
else if (a < 0) then
  if (b > 0.1) then
    ! Do other stuff
  end if
else
  ! Default option
end if
```

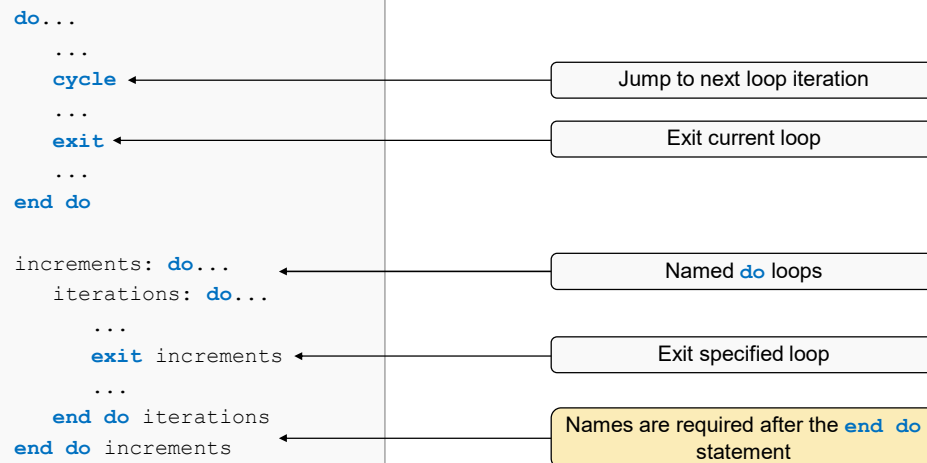
then statement is required

Parentheses are required and must
be Boolean

end if is required after block **if**

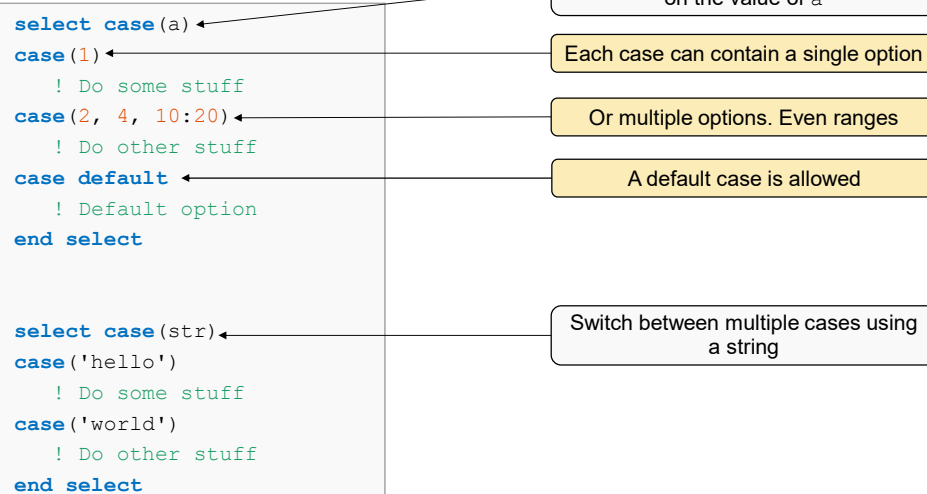
16

The Fortran language: Flow control: **do** loops



17

The Fortran language: Flow control: **select case**



18

The Fortran language: Derived data types (structures)

```
! Define a new data type
type Material
  real :: E, nu
  integer :: id
end type

! Variable declarations
type(Material) :: steel
real :: stress, strain

! Assign material parameters
steel%E = 200
...
! Recover stress
stress = steel%E*strain
```

Name of new data type

Derived data types can contain
multiple data fields of varying kinds

Indexing is done using the
percentage sign %

19

The Fortran language: Arrays: 1 dimensional (vectors)

```
! Vector with 8 elements
real, dimension(8) :: v

! Initialize with all zeros
v = 0

! Set first three elements
v(1:3) = (/1.0, 2.0, -41.9/)

! Add 3 to an element
v(7) = v(7) + 3.0
```

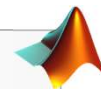


```
% Vector with 8 zeros
v = zeros(8,1);

% Initialize (not needed)
v(:) = 0;

% Set first three elements
v(1:3) = [1, 2, -41.9];

% Add 3 to an element
v(7) = v(7) + 3;
```



Array indexing in Fortran is very similar to MATLAB except for
minor syntax differences. Both languages use 1-indexing

20

The Fortran language: Arrays: 2 dimensional (matrices)



```
! 3x3 matrix
real, dimension(3,3) :: A

! Initialize with all zeros
A = 0

! Set element [1,2]
A(1,2) = 3.0

! Set second row
A(2,:) = (/1.0, 2.0, 3.0/)
```

```
% 3x3 matrix
A = zeros(3);

% Initialize (not needed)
A(:) = 0;

% Set element [1,2]
A(1,2) = 3;

% Set second row
A(2,:) = [1, 2, 3];
```



21

The Fortran language: Arrays: Dynamic

```
! Declare vector with unknown size
real, dimension(:), allocatable :: v

! Declare matrix with unknown size
real, dimension(:,:), allocatable :: A

! Allocate at run time
allocate(v(n))
allocate(A(n,m))

! Zero all elements
v = 0
A = 0
```

Declare arrays with sizes that are unknown at compile time

Allocate the arrays once the sizes are known at run time (n and m are integer variables)

22

The Fortran language: Arrays: Operations

```
! Vector dot product
x = dot_product(v1, v2)

! Matrix multiplication
C = matmul(A, B)

! Elementwise multiplication
C = A*B
```



```
% Vector dot product
x = dot(v1, v2);

% Matrix multiplication
C = A*B;

% Elementwise multiplication
C = A.*B;
```



Important to remember:
Fortran defaults to elementwise operations, MATLAB defaults to vector/matrix operations

23

The Fortran language: Output: Print

Try me!

```
program PrintTemp
  implicit none
  real :: T = 21.5

  ! Print using default formatting
  print*, T

  ! Print using a format specifier
  print'(f5.2)', T

  ! Print using default formatting
  print*, 'Temp. is ', T

  ! Print using a format specifier
  print'(Af5.2)', 'Temp. is ', T

end program
```

The asterisk * means default formatting. Often gives usable results

Prints

21.5000000

Print a floating point number with 2 decimals using at most 5 characters

Prints

21.50

Prints

Temp. is 21.5000000

Print a string of unspecified length followed by a floating point number with 2 decimals using at most 5 characters

Prints

Temp. is 21.50

24

The Fortran language: Output: Print

```

real, dimension(3) :: vec
real, dimension(2, 4) :: mat

...

! Print vec as row vector
print*, vec

! Print vec as column vector
print'(f20.8)', vec

! Print mat as matrix
print'(4f20.8)', transpose(mat)

```

Arrays are printed as rows when using default formatting

The format specifier is applied once per line printed, and thus the array is printed as a column

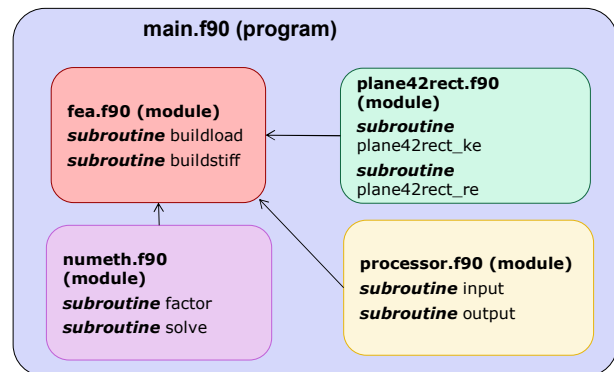
Fortran cycles through each element in each column first, but `print` displays rows. Transpose is needed for correct display of matrix

The format specifier can be applied multiple times per line printed to display a matrix. Here 4 times since the matrix has 4 columns

Let's get more advanced! How to structure programs as complexity increases



- Main program stored in the primary .f90 file
- Additional *procedures* stored in *modules* in separate .f90 files
- A module file can contain many variables, procedures and data types
- Data can be passed to procedures as arguments or made available multiple places using modules



Modules are essential to organizing Fortran projects. Think of them as libraries or packages where you can keep procedures, derived data types, variables to be used in multiple places, etc.

The Fortran language: Modules: A simple example

Try me!

```
mod.f90
module Constants
  implicit none
  real :: pi = 3.1415
end module
```

The module contains a single variable

```
main.f90
program MainProgram
  use Constants
  implicit none
  real :: area, radius

  radius = 2
  area = pi*radius**2
  print*, 'Area = ', area
end program
```

Make the module Constants available for use in the program

The parameter pi can now be used

27

The Fortran language: Procedures: Subroutines

Try me!

```
mod.f90
module CircleModule
  implicit none
  contains
  subroutine area(a, r)
    real :: a, r, pi = 3.141
    a = pi*r**2
  end subroutine
end module
```

Subroutine name and input/output arguments

If a subroutine returns a value, it simply modifies its arguments

```
main.f90
program MainProgram
  use CircleModule
  implicit none
  real :: a

  call area(a, 3.0)
  print*, 'Area = ', a
end program
```

Declare variable a

Call the subroutine. The subroutine updates the variable a

The variable a now contains the area

28

The Fortran language: Procedures: Argument intent

```

module NumTools
  implicit none
  contains

  subroutine Solver(A, b, i)
    real, intent(in) :: A
    real, intent(inout) :: b
    real, intent(out) :: i
    ...
  end subroutine
end module

```

A is transferred from the caller to the subroutine, where it is read-only. It is not returned to the caller

b is transferred from the caller to subroutine, where it can be overwritten and returned to the caller

i is not transferred from the caller. It is used only as an output and is returned to the caller

Specifying argument intent is not required but recommended. It is checked by the compiler which helps to find and eliminate mistakes

The Fortran language: Modules: Public or private

```

mod.f90
module NumTools
  implicit none
  private :: PreProc
  contains

  subroutine Solver(A, b, i)
    call PreProc(b)
    ...
  end subroutine

  subroutine PreProc(b)
    b = b + 2
  end subroutine
end module

```

Define the subroutine `PreProc` as private. It is only available *inside* the module

Define the subroutine `Solver`. By default everything in a module is public, so `Solver` is public

`Solver` can call `PreProc`

```

main.f90
program MainProgram
  use NumTools
  ...
  call Solver(A, b, i)
  call PreProc(b)
  print*, c
end program

```

OK

Error

Common Pitfalls

Quirks and Features

31

Quirks and Features: Integer division

Try me!

```
program IntegerPitfall
  implicit none

  ! Variable declaration
  real :: x
  x = 1.0

  ! Print results to screen
  print*, 2/3
  print*, 2/3*x
  print*, 2.0/3.0

end program
```

Dividing an integer by an integer
always produces an integer result!

Prints 0. Result is a truncated integer

Prints 0.000000. Result is a real but
still truncated

Prints 0.666666687

This behavior differs from MATLAB which
assumes all numeric variables to be double
precision floating point unless otherwise stated

32

Quirks and Features: Implicit declaration

Try me!

```
program AddTwoNumbers
  implicit none

  ! Variable declarations
  real :: m, n, k

  ! Divide two numbers
  m = 5.0
  n = 10.0
  k = m/n

  ! Print result to screen
  print*, 'The result is ', k

end program
```

Weird legacy quirk of Fortran:
Variable names starting with
i, j, k, l, m or n
are assumed to be **integer**, all
others assumed **real**.
implicit none
removes this assumption

Using **implicit none** allows the compiler
to detect missing declarations

33

Quirks and Features: Too small field width

Try me!

```
program main
  print' (f5.3) ', 12.345
  print' (f6.3) ', 12.345
  print' (f7.3) ', 12.345
  print*, 12.345
end program
```

Too small field width turns output into
asterisks.

Prints (without " ")
"*****"

Prints (without " ")
"12.345"

Prints (without " ")
" 12.345"

Prints (without " ")
" 12.3450003 "

Simple solution: Use default formatting

34

Quirks and Features: Do Loop Increments

Try me!

```
program FunnyDoLoop
  implicit none
  integer :: i

  do i = 1, 5
    print*, "spam"
  end do

  print*, "printed spam", i,"times"
end program
```

Do loops are incremented at the end of loop

Output:
" printed spam 6 times "

35

Quirks and Features: Wrong use of procedures

```
program ProcedureConfusion
  use MyModule
  implicit none

  ...

  a = 1 + call MyRoutine(x)

end program
```

Subroutines cannot be used inline in arithmetic

36

Quirks and Features: Implicit save

Try me!

```
mod.f90
module MyModule
  implicit none
  subroutine Sub
    implicit none
    integer :: a = 0
    a = a + 1
  end subroutine
end module
```

```
main.f90
program MainProgram
  implicit none
  call Sub
  call Sub
end program
```

Declaring and initializing a variable at the same time implicitly adds the save attribute

Initialization only happens once. The variable is saved between calls

Each call to the subroutine causes a to increment by 1

After this call a = 1

After this call a = 2

If the variable is not a constant, always separate the declaration and initialization steps

```
integer :: a
a = 0
```

37

Quirks and Features: Wrong dummy array shape

Specifying a *dummy shape* for an array in a subroutine can cause unexpected behavior!

```
main.f90
program MainProgram
  ...
  real, dimension(3,4) :: A
  call subBad(A)
  call subNice(A)
end program
```

```
mod.f90
module ArrayModule
  ...
  subroutine subBad(A)
    real, dimension(2,2) :: A
  end subroutine
  subroutine subNice(A)
    real, dimension(:, :) :: A
  end subroutine
end module
```

Let us assume

A =	1	1	1	1
	2	2	2	2
	3	3	3	3

Specifying the shape can cause implicit (without warning) reshape at run time such that inside the subroutine the matrix is

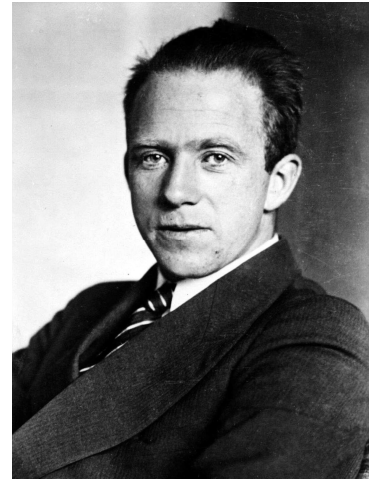
A =	1	3
	2	1

Using dynamic sizing in the input arguments is the safe way

38

Quirks and Features: The Heisen Bug

- Named after Werner Heisenberg
- Family of extremely nasty bugs
- Type of bug that alters behavior when attempting to study it
- Very difficult to trace
- Reproducibility is not always given



Other “fun” named bug types:

- Mandelbug
- Schrödingbug
- hindenbug
- Higgs-bugson

39

Quirks and Features: The Heisen Bug Uninitialized Variable

Try me!

```
program heisenbug_demo
  use stats
  implicit none
  real :: a(5) = [1.0, 2.0, 3.0, 4.0, 5.0]
  real :: clean, dirty, check

  clean = buggy_sum(a, 5)
  call other_work(check)
  print*, "hello"
  dirty = buggy_sum(a, 5)

  print *, 'Expected sum           = 15.0'
  print *, 'buggy_sum (clean run) =', clean
  print *, 'buggy_sum (after junk)=', dirty
end program heisenbug_demo
```

```
function buggy_sum(x, n) result(s)
  integer, intent(in) :: n
  real, intent(in)    :: x(n)
  real :: s, tmp
  integer :: i
  do i = 1, n
    tmp = tmp + x(i)
  end do
  s = tmp
end function buggy_sum
```

tmp has not been initialized

Print statement alters behavior

40

Quirks and Features: The Heisen Bug Indexing Out of Array

Try me!

```
program heisenbug_demo2
  use mystack
  implicit none
  type(stack_t) :: s
  integer :: k

  s%items = 0
  s%top = 0

  do k = 1, 5
    call push(s, k * 10)
  end do

  print *, 'top   =', s%top
  print *, 'total =', total(s)
end program
```

```
module mystack
  implicit none
  integer, parameter :: CAP = 4

  type stack_t
    integer :: items(CAP)
    integer :: top
  end type

  contains

  subroutine push(s, v)
    type(stack_t), intent(inout) :: s
    integer, intent(in) :: v
    s%top = s%top + 1
    s%items(s%top) = v
  end subroutine push

end module mystack
```

Memory is aligned, meaning
top is directly after items

Writing outside of array

41

Quirks and Features: Incorrect Procedure interface

Try me!

```
program incorrect_interface
  implicit none
  real :: values(4)
  real :: neighbouring_value
  values = 1.0
  neighbouring_value = 100.0
  ! Uncommenting this statement may change the result.
  ! print *, "Before call:", values
  call modify(values)
  print *, "Values:", values
  print *, "Neighbouring value:", neighbouring_value
end program incorrect_interface

subroutine modify(values)
  implicit none
  real :: values(10)
  values(10) = 2.0
end subroutine modify
```

Memory is aligned, meaning
top is directly after items

42

Quirks and Features: Non-Short-Circuit Logical operators

Try me!

```
program unsafe_bounds
  implicit none
  integer :: i
  integer :: values(5)
  values = [2, 4, 6, 8, 10]
  i = 6
  if (i <= size(values) .and.
     values(i) > 0) then
    print *, values(i)
  end if
end program unsafe_bounds
```

Short-Circuit is when the outcome of and if statement is determined before all expressions are evaluated

The compiler might change the order in which expressions are evaluated

Second expression could result in a sequence error

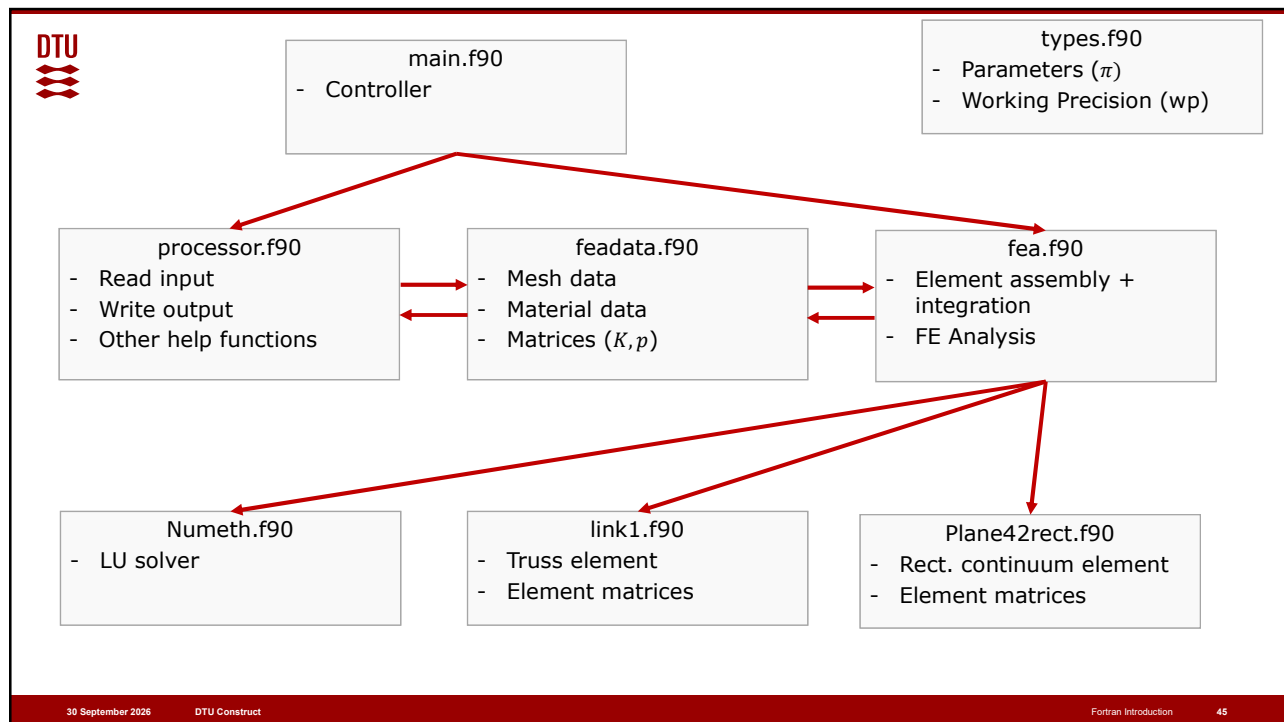
Safest to use nested evaluations:
If (i<=size(values)) then
 If (values(i)>0) then

43

FEM hand out script

Code Layout

44



45

Debugging and AI

Debugging

- Remember to read the compiler log, it often explains the problem ☺
- Print statements can be useful for debugging, but remember to clean your code occasionally

AI:

- Sometimes useful but most of the time a lot of pain
- Be very careful what changes it requests to make, especially in:
 - Numeth.f90
 - Processor.f90
 - Types.f90
 - Fedata.f90
- You almost never have to change these files, even if the Compiler and/or AI tells you the bug is there

30 September 2025 DTU Construct Fortran Introduction 46

46

Coding style

- Use meaningful variable, procedure and module names
 - **Good naming** increases readability of your code drastically
- Make your code **flexible**
 - Don't hard code material parameters, dimensions and other "magic numbers"
- Take the time to **clean up** your code after you have completed an assignment
 - Old clutter inevitably leads to bugs
- Use **subroutines** extensively to organize your code

47

MATLAB - Fortran conversion with details for this course

A MATLAB to Fortran conversion

General remarks and programming

	MATLAB
Editor	Built-in
Compilation	Run-time compilation
Cases	Case sensitive ($K \neq k$)
Variables	Scalar variables need not be
Dynamic allocation	NA
Modules	NA
Types	We did not use it
Subroutine calls	[areas] = bisect(areas,amin, function [areas] = bisect(ao :

The screenshot shows a file management interface with a list of files. A context menu is open over the file 'mat-f90', with the 'Download' option highlighted by a red rectangle. The file 'mat-f90' is also highlighted by a red rectangle. The interface includes options like 'Change File', 'Submit Feedback', 'Upload/Create', 'Move Up', 'Move Down', and 'Move To'. The file 'mat-f90' is a PDF document. The file 'mat-f90' is a PDF document. The file 'mat-f90' is a PDF document.

48